



Carátula para entrega de prácticas

Facultad de Ingeniería

Laboratorios de docencia

Laboratorio de Computación Salas A y B

Profesor(a): M.I. Edgar Tista García

Asignatura: Estructura de datos y algoritmos I

Grupo: 1

No de Práctica(s): 1

Integrante(s): Jiménez Olivo Evelin

No. de lista o 17
brigada:

Semestre: 2

Fecha de entrega: 14/02/2024

Observaciones:

CALIFICACIÓN: _____

Práctica 1. Arreglos

Objetivo Lab: Utilizar arreglos unidimensionales y multidimensionales para dar solución a problemas computacionales

Objetivo Clase: El alumno realizará aplicaciones en las que se utilicen arreglos de tipos de datos primitivos. Implementará apropiadamente el manejo de índices en arreglos multidimensionales

Previo

Descripción del previo

El programa es un código el cuál realiza la escítala espartana para cifrar y descifrar mensajes. Para esto se utilizaron tres funciones, las cuales son: la función principal main, la función “crearMensaje” y la función “descifrarMensaje”.

La función main contiene un bucle infinito que contiene un menú con tres opciones: crear mensaje cifrado, descifrar mensaje y salir. Dependiendo de la opción seleccionada, se llama a la función correspondiente.

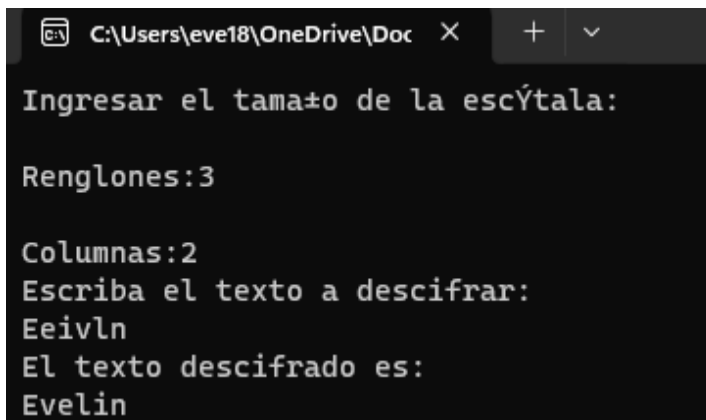
La función crearMensaje solicita al usuario el número de renglones y columnas del mensaje cifrado, estos se guardan en arreglos bidimensionales y por medio de un arreglo se multiplican los renglones por las columnas para así obtener el tamaño del texto, luego se le pide ingresar un texto a cifrar, para cifrar el texto se hace uso de ciclos for, para primero contar los renglones y después las columnas. Después se cifra el mensaje para esto se hace igualmente uso de ciclos for pero ahora se cuentan primero las columnas y luego los renglones. Finalmente se imprimen en arreglos.

En este programa se asume que el texto de entrada no contiene espacios, ya que se utiliza `scanf("%s", texto)` para leer el texto.

La función descifrarMensaje es similar a la función crearMensaje, solicita al usuario el número de renglones y columnas del mensaje cifrado, estos se guardan en arreglos bidimensionales, se multiplican los renglones por las columnas para así obtener el tamaño del texto, se pide el texto a descifrar y en pantalla se muestra el texto descifrado.

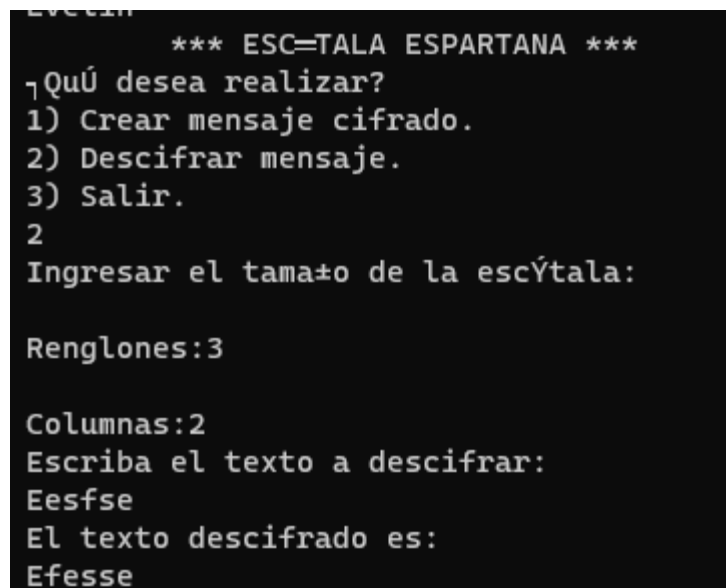
La salida del programa es un menú, en el cual se le pregunta al usuario que quiere realizar, por lo que se le presentan 3 opciones. La primera es cifrar mensaje, la segunda es descifrar mensaje y la tercera es salir.

Si el usuario elige la opción 1, inmediatamente se le pide el tamaño del texto, es decir los renglones y las columnas, posterior a que lo ingresa se le pide el texto que desea cifrar, cuando lo ingresé enseguida le mandará el mensaje cifrado y vuelve a aparecer el menú.



```
C:\Users\eve18\OneDrive\Doc X + v
Ingresar el tamaño de la escÿtala:
Renglones:3
Columnas:2
Escriba el texto a descifrar:
Eeivln
El texto descifrado es:
Evelin
```

Si el usuario elige la opción 2, es muy parecida a la opción 1, se pide el tamaño y después se le pide que ingrese el mensaje que quiere descifrar, después de ingresarlo, se muestra el mensaje ya descifrado y vuelve a aparecer el menú.



```
Evelin
*** ESC=TALA ESPARTANA ***
¿Qué desea realizar?
1) Crear mensaje cifrado.
2) Descifrar mensaje.
3) Salir.
2
Ingresar el tamaño de la escÿtala:
Renglones:3
Columnas:2
Escriba el texto a descifrar:
Eesfse
El texto descifrado es:
Efesse
```

Si se elige la opción 3 se finaliza el programa.

```
*** ESC=TALA ESPARTANA ***
¿Qué desea realizar?
1) Crear mensaje cifrado.
2) Descifrar mensaje.
3) Salir.
3
```

El programa utiliza arreglos bidimensionales para representar la escítala y manipular el texto de entrada según las reglas de la escítala espartana. En los arreglos se guarda el texto que se desea cifrar para posteriormente ser descifrado, es decir los arreglos funcionan como memoria de almacenamiento.

EJERCICIOS PROPUESTOS PARA EL DESARROLLO DE LA PRÁCTICA

Ejercicio 1.- Elabora un programa con las siguientes instrucciones

El programa lo que hace es primero declarar arreglos unidimensionales con variables de distinto tipo, los cuales tienen inicialización, luego con el printf se imprime el carácter indicado del índice de los arreglos de tipo de char.

```
o
a
a
l
```

a) Agrega instrucciones para imprimir elementos de los arreglos a b y c

Para poder imprimir los elementos de los arreglos a,b y c se podía de dos formas una imprimir elemento por elemento con varios printf o la segunda como yo lo realice la cual fue mediante ciclos for imprimir los elementos. Los ciclos for se inicializaban en 0 y finalizaban en el último índice del arreglo, ya después dentro de los ciclos for con un printf se tenía acceso a los arreglos y de esa manera se imprimían los elementos

```
18     for(i=0; i<=3; i++)
19     {
20         printf("%i\n",a[i]);
21     }
22     for(i=0; i<=3; i++)
23     {
24         printf("%.2f\n",b[i]);
25     }
26     for(i=0; i<=2; i++)
27     {
28         printf("%i\n",c[i]);
29     }
```

```
14
21
28
35
10.50
11.33
14.44
77.33
100
200
300
```

b) Que ocurre al imprimir posiciones “fuera del rango”

El imprimir posiciones “fuera del rango” es un error lógico ya que el querer imprimir esas posiciones significa que se está intentando acceder a un elemento del arreglo que está más allá de los límites definidos para ese arreglo. Por lo que al acceder a una posición fuera del rango definido en el arreglo puede causar un comportamiento indefinido ya que los arreglos constan de memorias contiguas el compilador seguirá ejecutando y nos imprimirá lo que se encuentre en esa dirección de memoria, pero lo que se imprima o se realice al hacer esa acción depende del compilador ya que por ejemplo en Linux por alguna razón se encontraba en la memoria primero la

cadena 3 y luego la cadena dos por lo que al querer imprimir posiciones fuera del rango de la cadena 3 , nos imprimía elementos de la cadena 2. Sin embargo, en dev c al salir del rango no imprime nada, solo imprime el espacio.

```
16 | printf("%c \n", cad3[6]);
```

Linux:

```
o
bc20d70d
bc20d70e
bc20d70f
bc20d710
```

Dev C

```
o
a
a
```

Por lo que en la línea 16 del código tuve que realizar una modificación e imprimir un elemento que estuviera dentro del tamaño del arreglo

```
15 | printf("%c \n", cad3[3]);
16 | printf("%c \n", cad3[2]);
```

```
o
a
a
l
```

Para poder realizar este programa se necesitan de conocimientos teóricos como que es un arreglo, sintaxis de un arreglo, la inicialización de arreglos, el acceso a elementos de arreglos y el manejo práctico de ellos.

Los conceptos se aplican al declarar e inicializar los arreglos, al saber que sucede si se imprime fuera del rango del tamaño, esto era fundamental para así solucionar el error lógico que había y al imprimir cierto elemento de los arreglos.

c) Realiza los comentarios acerca de los print de arreglos de caracteres y realiza los comentarios correspondientes

La función que tiene los print de los arreglos de caracteres es que se accede al elemento del índice indicado ahí y los imprime

En este ejercicio se logro resolver un 100% de acuerdo a los requisitos solicitados

Para poder resolver este problema lo primero que hice fue analizar todo el código después codifique, compile y ejecute para ver que sucedía. Tuve un pequeño problema al entender porque al excederse fuera del rango del arreglo 'cad3' se imprimían elementos del arreglo 'cad2', pero después con la teoría ya vista y el realizar diferentes pruebas entendí el porqué.

Ejecución:

```
o
a
a
l
14
21
28
35
10.50
11.33
14.44
77.33
100
200
300
```

```
-----
```

```
Process exited after 12.21 seconds with return value 0
Presione una tecla para continuar . . .
```

Ejercicio 2

a) Codifica, compila y ejecuta el siguiente código, indica la salida y qué hace el programa

Lo primero que hace el programa es inicializar un arreglo de 10 elementos. Después declara dos variables de tipo entero “valor” y “i” las cuales las inicializa en 0. Luego mediante un ciclo do while se imprime el primer elemento del arreglo, posteriormente mediante la siguiente instrucción:

```
valor+=lista[i];
```

Agrega el valor del elemento actual de arreglo al valor actual de valor. Es decir, suma el elemento actual del arreglo al acumulador valor. Luego se aumenta en 1 a i para así acceder a cada elemento del arreglo secuencialmente. El arreglo se ejecuta hasta que i sea menor que C (10).

Finalmente se imprime todos los elementos del arreglo, más la suma total de ellos.

```
10
20
30
40
50
60
70
80
90
100
El valor es: 550

-----
Process exited after 19.39 seconds with return value 20
Presione una tecla para continuar . . .
```

b) Modifica el programa anterior de tal forma que en el valor de salida solamente se sumen los elementos de las posiciones “impares” del arreglo (1,3,5,7, etc).

Lo que hice para que se sumara solo los elementos de las posiciones “impares del arreglo fue:

1.- Asignarle a la variable i un valor de 1 en lugar de un valor de 0, para que así se empezará desde la posición 1.

```
6 | int valor=0, i=1;
```

2.- Posteriormente fue cambiar el incremento de la variable i, en lugar de que se incrementara en 1, lo que hice fue que incrementara en 2.

11

```
i=i+2;
```

Ejecución 1:

```
int lista[C]={10,20,30,40,50,60,70,80,90,100};
int valor=0; i=1;
```

```
20
40
60
80
100
El valor es: 300
```

Ejecución 2:

```
int lista[C]={10,15,30,25,50,102,70,256,90,78};
```

```
15
25
102
256
78
El valor es: 476
```

Para poder realizar este programa se necesitan de conocimientos teóricos como inicialización de arreglos y cómo funciona el ciclo do while y los contadores.

Los conceptos se aplican al declarar e inicializar los arreglos, al acceder a los elementos del arreglo e imprimirlos para que posteriormente cada uno se sume.

```
int valor=0; i=0;
do
{
    printf("\n %d", lista[i]);
    valor+=lista[i];
    i++;
} while(i<C);
```

En este ejercicio el avance que se logró resolver fue de un 100%

Para poder resolver este ejercicio la lógica que utilice fue analizar las instrucciones que había dentro del ciclo do while para saber qué era lo que sucedía con el arreglo, ya que lo entendí, procedí a realizar la modificación de imprimir las posiciones impares y sumarlas, para esto analice que si la primera posición impar era 1, tenía que inicializar la variable i en 1 y de cada posición impar hay una diferencia de dos, tenía que cambiar el incremento a 2.

Ejercicio 3.-

Indica los errores del siguiente programa (línea, explicación del error), posteriormente codifica el programa con las correcciones respectivas

Lo que el programa hace es imprimir los valores de un arreglo de 3*3 el cual se inicializo, cada elemento lo imprime en forma de lista, por medio de 3 ciclos for para un manejo práctico de los arreglos.

Luego hay un ciclo for el cual no tiene la instrucción de imprimir la operación que está dentro de él, pero si se imprime, se imprimen los valores de 3 al 12, pero si no se imprime esa instrucción, no afecta en nada al programa por lo que lo podemos quitar.

Imprimiendo el ciclo for:

```
18 for (i=0; i<10; i++)
19 {
20     a1[i]=i+3;
21     printf("%d \n", a1[i]);
22 }
```

```
3
4
5
6
7
8
9
10
11
12
mito
```

Después hay 4 printf que lo que hacen es imprimir cierto elemento del arreglo char, al imprimir estos printf se forma la palabra "m i t o".

```
mito
```

Los errores que hubo en el programa son:

Línea 4: Declarar la variable j que se utiliza para un ciclo for

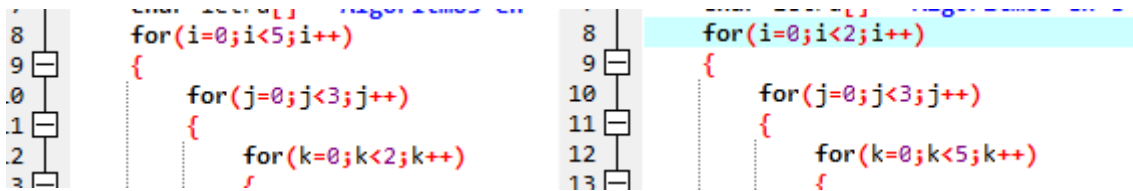
Línea 6: Falta el punto y coma en esa instrucción.

Línea 7: Como al final del programa se está pidiendo imprimir los elementos de ese arreglo fuera del valor que está establecido y como su tamaño es mayor a 5 podemos quitar el tamaño de ese arreglo y dejar los corchetes vacíos.

Línea 8 y 12: intercambiar los valores finales de los ciclos for de esa línea, ya que el ciclo for externo debe corresponder al primer índice y los ciclos internos a los últimos dos arreglos.

Error:

Corregido:



Línea 20: colocar bien la variable ya que se manda a llamar a la variable "a" y se tiene que mandar a llamar a la variable "a1"

Línea 24-27: Se está imprimiendo el arreglo 'letra' como tipo entero cuando se debe imprimir como tipo char por lo que se debe cambiar "%d" a "%c".

c) Dibuja en papel el arreglo que se genera en el programa anterior

Arreglo que se imprime

	0	1	2	3	4
0	2	4	6	8	10
1	12	14	16	18	20
2	22	24	26	28	30

	0	1	2	3	4
0	32	34	36	38	40
1	42	44	46	48	50
2	52	54	56	58	60

Los conocimientos teóricos que se necesitan para corregir este programa es saber cómo imprimir cada variable ya que se debe corregir el error de cuando se imprime un arreglos de tipo char como entero, el correcto manejo práctico de los arreglos multidimensionales para poder declarar bien los ciclos anidados y como inicializar un arreglo de tipo char para que así no se genere un error lógico cuando sus elementos son mayor al tamaño que se indica.

En este ejercicio el avance que se logró resolver fue de un 100%

Hubo una dificultad que era saber que función hacia el siguiente for:

Por lo que hice varias pruebas y me di cuenta de su función.

Ejecución:

```
C:\Users\eve18\OneDrive\Doc
Arreglo[0][0][0]: 2
Arreglo[0][0][1]: 4
Arreglo[0][0][2]: 6
Arreglo[0][0][3]: 8
Arreglo[0][0][4]: 10
Arreglo[0][1][0]: 12
Arreglo[0][1][1]: 14
Arreglo[0][1][2]: 16
Arreglo[0][1][3]: 18
Arreglo[0][1][4]: 20
Arreglo[0][2][0]: 22
Arreglo[0][2][1]: 24
Arreglo[0][2][2]: 26
Arreglo[0][2][3]: 28
Arreglo[0][2][4]: 30
Arreglo[1][0][0]: 32
Arreglo[1][0][1]: 34
```

```
Arreglo[1][0][2]: 36
Arreglo[1][0][3]: 38
Arreglo[1][0][4]: 40
Arreglo[1][1][0]: 42
Arreglo[1][1][1]: 44
Arreglo[1][1][2]: 46
Arreglo[1][1][3]: 48
Arreglo[1][1][4]: 50
Arreglo[1][2][0]: 52
Arreglo[1][2][1]: 54
Arreglo[1][2][2]: 56
Arreglo[1][2][3]: 58
Arreglo[1][2][4]: 60
```

```
3
4
5
6
7
8
9
10
11
12
mito
-----
Process exited after 12.13 seconds with return value 111
Presione una tecla para continuar . . . |
```

Ejercicio 4. Escribe un programa para trabajar sobre un arreglo unidimensional de 10 elementos en el cual se mostrará al usuario un menú con las siguientes operaciones:

- 1) Ingresar o modificar elementos del arreglo
- 2) Mostrar el resultado de la suma de los elementos
- 3) Mostrar el resultado de la multiplicación de los elementos
- 4) Realizar la suma solo de los elementos divisibles entre 4
- 5) Multiplicar cada elemento del arreglo por un número dado por el usuario

Lo que hace el programa primero se define un arreglo de tamaño 10

Después se define una variable global la cual es `i` y que se utilizara en todos los ciclos for, posteriormente hay 4 funciones:

En la función “Suma” lo que se hace primero es declarar una variable `suma` la cual se le asigna un valor de 0 y a esta variable se utiliza para almacenar la suma acumulada de todos los elementos del arreglo. Después se hace uso de un ciclo for para acceder a cada elemento del arreglo y recorrer la posición de este, después mediante una instrucción se suma el valor del elemento actual del arreglo mas la variable `suma`. Finalmente, con el `return` se devuelve el valor acumulado en “suma”

```
for (i = 0; i < Arreglo; i++)  
{  
    suma += arreglo[i];  
}  
return suma;
```

En la función “Multiplicación sucede lo mismo que en la función “Suma” la única diferencia es que se declara una variable “`mult`” a la cual se le asigna un valor de 1, y en ves de hacer una operación suma, se realiza una operación multiplicación.

```
int mult=1;  
for (i = 0; i < Arreglo; i++)  
{  
    mult *= arreglo[i];  
}
```

En la función “Divisible4” se hace algo parecido a las anteriores funciones solo que aquí le agregamos un `if` el cual evalúa. Si el elemento actual del arreglo es divisible entre 4, si el residuo es igual a 0, se suman cada elemento del arreglo que es divisible entre.

```

        if (arreglo[i] % 4 == 0)
        {
            valor += arreglo[i];
        }
    }

```

Finalmente se encuentra la función principal main, en esta función por medio de un do while se realiza el menú para que se pueda imprimir hasta que el usuario decida salir. Después por medio de un switch se realiza cada acción del menú, en el case 1 se piden los elementos del arreglo y lo imprime, después en los case 2,3 y 4 se mandan a llamar las funciones “Suma”, “Multiplicacion” y “Divisible4” respectivamente y se imprime el arreglo y los resultados.

```

        printf("\n El valor es: %d \n",Divisible4(arreglo));
    }

```

Después en el case 5: Se multiplica por un número cada elemento del arreglo y por último en el case 6 se sale del programa.

Los elementos teóricos que se necesitaban es el correcto manejo de los ciclos for y do while, de las funciones, así como el correcto manejo de los arreglos (acceder a sus elementos, hacer operaciones con sus elementos, etc).

La aplicación de los conceptos de arreglos se aplica cuando se pide ingresar los 10 elementos del arreglo.

```

    case 1:
        printf("Ingrese o modifique los elementos del arreglo:\n");
        for (i = 0; i < 10; i++)
        {
            printf("Ingrese el elemento %d: ",i+1);
            scanf("%d", &arreglo[i]);
        }
    }

```

Cuando en las funciones se acceden a ellos y se realizan operaciones con sus elementos, cuando se hace un manejo práctico con los ciclos for para que se puedan imprimir correctamente.

```

    int mult=1;
    for (i = 0; i < Arreglo; i++)
    {
        mult *= arreglo[i];
    }
    return mult;
}

```

La estrategia para resolver este problema primero era como imprimir el menú ya que lo realicé, con el switch pensé en como pedir los elementos de un arreglo, por lo que decidí utilizar ciclos for y mandar a imprimir los arreglos correctamente. Después hice las operaciones, de ahí me guié en el ejercicio 2 de la práctica algunas dudas que tuve, al principio las había colocado sin funciones, pero me surgió un

problema que al querer acceder de nuevo en una opción me multiplicaba el valor por lo que al utilizar funciones podía devolver el valor correcto de la operación.

En este ejercicio el avance que se logro resolver fue de un 100%

Ejecución:

```
MENU:
1) Ingresar o modificar elementos del arreglo
2) Mostrar el resultado de la suma de los elementos
3) Mostrar el resultado de la multiplicaci n de los elementos
4) Realizar la suma solo de los elementos divisibles entre 4
5) Multiplicar cada elemento del arreglo por un numero dado por el usuario
6) Salir
Ingrese su opcion: 1
Ingrese o modifique los elementos del arreglo:
Ingrese el elemento 1: 1
Ingrese el elemento 2: 2
Ingrese el elemento 3: 3
Ingrese el elemento 4: 4
Ingrese el elemento 5: 5
Ingrese el elemento 6: 6
Ingrese el elemento 7: 7
Ingrese el elemento 8: 8
Ingrese el elemento 9: 9
Ingrese el elemento 10: 10
Arreglo: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

```
MENU:
1) Ingresar o modificar elementos del arreglo
2) Mostrar el resultado de la suma de los elementos
3) Mostrar el resultado de la multiplicaci n de los elementos
4) Realizar la suma solo de los elementos divisibles entre 4
5) Multiplicar cada elemento del arreglo por un numero dado por el usuario
6) Salir
Ingrese su opcion: 2
Arreglo: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

El valor es: 55

MENU:
1) Ingresar o modificar elementos del arreglo
2) Mostrar el resultado de la suma de los elementos
3) Mostrar el resultado de la multiplicaci n de los elementos
4) Realizar la suma solo de los elementos divisibles entre 4
5) Multiplicar cada elemento del arreglo por un numero dado por el usuario
6) Salir
Ingrese su opcion: 3
Arreglo: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

El valor es: 3628800
```

```

MENU:
1) Ingresar o modificar elementos del arreglo
2) Mostrar el resultado de la suma de los elementos
3) Mostrar el resultado de la multiplicaci n de los elementos
4) Realizar la suma solo de los elementos divisibles entre 4
5) Multiplicar cada elemento del arreglo por un numero dado por el usuario
6) Salir
Ingrese su opcion: 4
Arreglo: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

El valor es: 12

MENU:
1) Ingresar o modificar elementos del arreglo
2) Mostrar el resultado de la suma de los elementos
3) Mostrar el resultado de la multiplicaci n de los elementos
4) Realizar la suma solo de los elementos divisibles entre 4
5) Multiplicar cada elemento del arreglo por un numero dado por el usuario
6) Salir
Ingrese su opcion: 5
Ingrese un numero para multiplicar cada elemento del arreglo: 2
Arreglo: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
Arreglo multiplicado por 2: [2, 4, 6, 8, 10, 12, 14, 16, 18, 20]

```

Ejercicio 5.

Convirtiendo a Binario Escribe un programa en el cual se solicite al usuario llenar un arreglo de 12 elementos ( nicamente 0's y 1's). Una vez que el usuario llene el arreglo con el n mero binario, podr  seleccionar entre alguna codificaci n

a) Binario puro

b) Punto fijo (6 bit entero y 6 bit decimal)

c) Representaci n de complemento a 2

Lo que hace este programa es que cuando el usuario llene los elementos de un arreglo de 12 elementos en forma binaria elija mediante un men  de que manera quiere convertir ese binario en forma decimal. Puede ser por binario puro, representaci n punto fijo o representaci n complemento A2.

Por medio de 3 funciones se realiza este programa:

La primera es la funci n "binarioPuro" lo cual declara una variable la cual se inicializa en un valor de 0 y esta variable se utiliza para almacenar el valor decimal. Despu s mediante un ciclo for se recorre el arreglo de manera inversa siendo su valor inicial el  ndice 11 y el valor final el  ndice 0, ya que para convertir en binario no se hace de izquierda a derecha si no de derecha a izquierda. Despu s mediante una instrucci n se calcula el valor decimal del bit actual multiplicando el valor del bit del arreglo por 2 elevado a la potencia correspondiente.

```

decimal += arreglo[i] * pow(2, 12 - 1 - i);

```


Porque se devuelve el valor completo sumando la parte entera con la parte decimal dividida por 64, el cual es 2 elevado a la 6), lo que representa el desplazamiento de la coma hacia la derecha para obtener la parte decimal.

En la tercera función “complementoA2” primero se evalúa si el primer elemento del arreglo es 1 ya que si es así se realiza el complemento del binario mediante un ciclo for el cual cambia cada bit del número binario a su complemento. Si un bit es 1, se cambia a 0; si es 0, se cambia a 1, mediante un operador ternario.

```
for (i = 0; i < 12; i++)  
{  
    arreglo[i] = (arreglo[i] == 0) ? 1 : 0;  
}
```

Si el primer elemento del arreglo es 0 se llama a la función “binarioPuro”

En la función main, se obtienen las instrucciones para imprimir los 12 elementos del arreglo, se realiza el menú mediante un do while y un switch, posteriormente en los casos del switch se mandan a llamar las funciones correspondientes. Para el “puntoFijo” se utiliza un ciclo for el cual recorre los primeros 6 bits y en cada iteración se calcula parte entera en forma decimal, por lo que se utiliza instrucción parecida a la anterior para calcular el valor decimal del bit actual. Con el otro for se calcula la parte decimal.

Para realizar el programa se necesitaba de conocimientos teóricos como el correcto manejo de funciones, de ciclos for y do while, esto para acceder correctamente a cada elemento y bit del arreglo ya que para poder convertir a decimal en cada una de las opciones se necesitaba acceder al elemento correcto y hacer la iteración correcta para que así el resultado que se imprimiera fuera correcto.

Para realizar este problema tuve varias complicaciones al realizar el complemento A2 ya que no sabía exactamente como hacer el complemento cuando era negativo por lo que investigué y encontré el operador ternario para poder realizarlo.

Considero que el avance que logré en este ejercicio fue de un 90%, considero que me faltaron puntos para hacerlo más eficiente

Ejecución:

Ingresa 12 elementos (unicamente 0's y 1's)

Ingrese el elemento 1: 1

Ingrese el elemento 2: 0

Ingrese el elemento 3: 0

Ingrese el elemento 4: 1

Ingrese el elemento 5: 0

Ingrese el elemento 6: 1

Ingrese el elemento 7: 0

Ingrese el elemento 8: 1

Ingrese el elemento 9: 0

Ingrese el elemento 10: 0

Ingrese el elemento 11: 1

Ingrese el elemento 12: 1

Arreglo: [100101010011]

Seleccione la opcion:

a) Binario puro

b) Punto fijo (6 bits enteros y 6 bits decimales)

c) Representacion de complemento a 2

d) Salir

b

El resultado en decimal es: 37.296875

Seleccione la opcion:

a) Binario puro

b) Punto fijo (6 bits enteros y 6 bits decimales)

c) Representacion de complemento a 2

d) Salir

a

El resultado en decimal es: 2387

Seleccione la opcion:

a) Binario puro

b) Punto fijo (6 bits enteros y 6 bits decimales)

c) Representacion de complemento a 2

d) Salir

c

El resultado en decimal es: -1709

Conclusiones:

Si se cumplieron los objetivos ya que para realizar los ejercicios 4 y 5 se necesitaba el correcto manejo y funcionamiento de los arreglos, a lo mismo que en los problemas 1, 2 y 3 se utilizaban arreglos unidimensionales y multidimensionales para dar solución a los problemas computacionales.

Se contribuyo al aprendizaje del concepto visto ya que me pude dar cuenta como se puede realizar un manejo practica de los arreglos, a parte aprendí a la correcta iteración para poder imprimir los elementos correctos y así elaborar el programa o corregir los errores que había en ellos. Igualmente aprendí a pedir los elementos al usuario y a guardarlos en el arreglo, ya que eso siempre se me dificultaba y se me hacía complicado realizarlo.

La critica que puedo hacer es mejorar en las instrucciones de los ejercicios, ya que suelen ser confusas en algunas ocasiones.

El porcentaje de avance global fue de un 95% ya que se logró concluir la mayoría de ellos en un 100%, sin embargo, en el último le hicieron falta detalles y optimizarlo para hacerlo mas eficiente y entendible